A HIGH LEVEL DESIGN

ON

MAS: A Multi-Agent Software Engineering Team for Automated Code Generation

Submitted in the partial fulfillment of requirements to

CS 454 – Project Lab

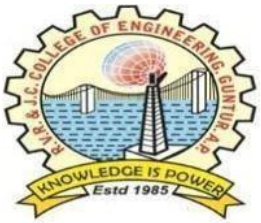
By

Batch No. 20

KOLLI HIMARAGA VARSHITH (Y23CS072)

KONGARA SRI VARSHITHA (Y23CS075)

PASUPULETI SRAVANI (Y23CS132)



R.V.R. & J.C. COLLEGE OF ENGINEERING (Autonomous)

(NAAC 'A+' Grade)

Approved by AICTE :: Affiliated to Acharya Nagarjuna University

Chandramoulipuram :: Chowdavaram

GUNTUR – 522019, Andhra Pradesh, India.

Sri. S Karthik	**Dr. Z Sunitha Bai**	**Prof N Nagamalleswara Rao**
Project Guide	Project In-Charge	Prof. & Head, CSE

Contents

1 Problem Statement **3**

1.1 Why We Need MAS 3

2 Functional Requirements **5**

3 Basic Architecture **7**

3.1 Major Components 7

3.1.1 Major Modules 8

3.2 Architecture Diagram 8

3.2.1 Deployment View 9

3.3 Data Flow Diagrams 10

3.3.1 DFD Level 0 (Context Diagram) 10

3.3.2 DFD Level 1 10

3.4 Workflows 12

3.4.1 Overall Workflow 12

3.4.2 Individual Workflows 14

3.5 Architectural Patterns 15

4 Non Functional Requirements **17**

5 Technology Stack Used **18**

6 Summary **19**

1. Problem Statement

These days, large language models (LLMs) can write small programs from a simple description in English. Some tools make many LLM agents work together like a small software team. One agent understands the requirement, one designs the software, one writes the code and one reviews it. ChatDev, MetaGPT and iCoder are examples of such tools. They give better code than a single LLM call. But they also cost a lot. Every task goes through every stage, every stage gets the whole project as input, and every review is done by an LLM. Because of this, a large number of tokens is used.

The base paper for our project is iCoder (You et al., 2026). It gives the best quality among the tools it was compared with. But it uses about 104,883 tokens per task with GPT-4o, while ChatDev uses only about 30,417. With GPT-3.5 Turbo it uses about 411,145 tokens per task, while ChatDev uses about 22,327. The paper also leaves some gaps:

- Quality is measured only by completeness, executability and consistency. Security and maintainability are not checked.
- The results are not repeated and no significance test is done.
- Changing requirements and failures of agents are not tested.
- Nobody checks whether the extra agents and extra rounds are worth their cost.

The goal of this project is to design, build and test **MAS (Multi-Agent Software Engineering Team)**. MAS keeps the good parts of structured development, like architecture design, dependency checking and review rounds, but it tries to use fewer model calls and fewer tokens. The main ideas are:

- Simple tasks take a fast path and skip the design stage.
- Structure checks are done by normal code (AST analysis) and not by an LLM.
- Only the changed files are sent in the next round.
- The loop stops when a new round does not improve the code enough.
- Every output of an agent is checked before the next agent uses it.

We compare MAS directly with ChatDev, using the same tasks, the same models and the same scoring scripts. The source code of iCoder is not available, so iCoder is used only through the results published in its paper.

1.1 Why We Need MAS

Problem seen in iCoder	**What MAS does**
Every task goes through all four agents (RABot, UMLBot, DevBot, ReviewBot)	A router checks the task first. Simple tasks skip the design and sprint stages.
Dependency review is done by an LLM	AST analysis finds cycles and missing imports without using any tokens.
The full context is sent in every round	Only the changed files and the needed part of the UML are sent.
Number of rounds is fixed and cost is not studied	The loop stops when the gain is small. We report quality per 1,000 tokens.
Quality is only completeness $\times$ executability $\times$ consistency	We also measure Ruff, Bandit, Radon and a neutral shared test.
No repeated runs and no significance test	Three repeats, mean and standard deviation, and a paired permutation test.
Changing requirements and agent failures are not tested	We add a change-request flow, schema checks, rollback and small fault tests.

2. Functional Requirements

The functional requirements tell what MAS must be able to do. MAS should take a requirement in plain English, produce working code, check the code, and let us measure how good and how costly the result is.

1. **Taking the requirement:** The user types a software requirement in plain English using the CLI or the API. The user also chooses the model.
2. **Preparing the task:** The Manager agent cleans up the request and creates the run and the shared state.
3. **Checking the complexity:** The Router gives the requirement a score using fixed rules. It does not call any LLM. The task becomes SIMPLE or STRUCTURED.
4. **Designing only when needed:** For STRUCTURED tasks the Architect makes a small UML design and a sprint plan. SIMPLE tasks skip this step.
5. **Writing the code:** The Developer agent writes the Python code. It gets only the task, the changed files and the part of the UML that it needs.
6. **Running the code safely:** All generated code runs inside a Docker container. The container has no network and has limits on CPU, memory and time.
7. **Checking the structure:** An AST analyser builds the import graph. It finds circular imports and missing imports and tells which modules are affected by a change.
8. **Checking the quality:** Ruff checks the style, Bandit checks the security and Radon checks how easy the code is to maintain.
9. **Testing:** The Tester agent writes pytest tests and records the pass rate and the coverage. MAS uses these only for itself and not for comparing with ChatDev.
10. **Reviewing the meaning:** The Reviewer agent uses the cheaper GPT-4o-mini model. It checks whether the code matches the requirement.
11. **Deciding when to stop:** The Reflection agent calculates a score after every round. It stops when the code is clean, when the gain is very small, or when the round limit is reached.
12. **Checking agent outputs:** Every output that goes from one agent to another is checked against a JSON schema. If it is wrong, it is retried once. If it is still wrong, we go back to the last good checkpoint.
13. **Handling changes:** When the requirement changes, MAS finds the affected files, updates only those files, runs the regression tests and looks for code that is no longer needed.

14. **Counting tokens:** Every LLM call goes through a logging proxy. It saves the input tokens, output tokens, time taken and the cost.

15. **Running the benchmark:** MAS loads the SRDD tasks with a fixed seed and runs MAS and ChatDev on the same tasks. A hidden neutral evaluator scores both and the statistics are calculated.

16. **Ablation study:** Each saving idea (router, AST analysis, context slicing, cheap reviewer, adaptive stop, UML and sprint) can be switched off one by one.

17. **Live view:** Events of the agents are sent over a WebSocket to a small React page. The CLI also prints them.

18. **Handling bad input:** MAS detects empty, unclear or contradicting requirements, model time-outs and broken model output. It shows a clear error and a fallback.

3. Basic Architecture

3.1 Major Components

1. **Manager Agent:** This is the starting point of the workflow. It cleans up the requirement, creates the run and the shared state, and controls the other agents.
2. **Complexity Router:** It scores the requirement text using keyword and regex rules. A score of 0 to 3 means SIMPLE and 4 or more means STRUCTURED. It costs no tokens.
3. **Architect Agent:** It is used only for STRUCTURED tasks. It makes a small UML design and a sprint plan. Later it gives the Developer only the UML part that is needed.
4. **Developer Agent:** It writes the first version of the code and the later patches. It gets only the task, the changed files and the needed UML part.
5. **Tester Agent:** It writes pytest tests and records the pass rate and the coverage. These help MAS decide when to stop.
6. **Reviewer Agent:** It runs on GPT-4o-mini. It checks the meaning of the code and whether the requirement is met, using the AST, test and tool reports.
7. **Reflection Agent and Adaptive Stopper:** It calculates the score of each round and the gain from the last round. Then it decides to continue or stop.
8. **AST Dependency Analyser:** It reads the changed Python files and builds the import graph. It finds cycles and missing imports and shows the impact of a change. No LLM is used.
9. **Quality Tools:** Ruff, Bandit and Radon, run with the same commands and settings for every project.
10. **Schema Validator and Checkpoint Manager:** The validator checks every artifact against its JSON schema. The checkpoint manager saves Git checkpoints so that we can roll back.
11. **Docker Sandbox:** It runs the generated code with no network, one CPU, 512 MB memory, a limited number of processes and a time limit of 30 to 60 seconds.
12. **LLM Provider and Logging Proxy:** One interface for GPT-4o and GPT-4o-mini. The proxy records the usage for both MAS and ChatDev, so tokens are counted in the same way.
13. **Evaluation Harness:** SRDD loader, ChatDev adapter, the hidden neutral evaluator, metric scripts, ablation switches and the statistics code.
14. **Data Layer:** PostgreSQL stores runs, events, artifacts, checkpoints, model calls and metrics. The workspace and Git hold the generated code.

15. **Interfaces:** A CLI as the main interface and one small React and Tailwind page with a live agent log, a results table and a token and cost chart.

3.1.1 Major Modules

Module	Parts	Work it does
User Module	CLI, React page, REST and WebSocket client	Send requirements and change requests, watch runs, see results
Agent Module	Manager, Router, Architect, Developer, Tester, Reviewer, Reflection	Plan, write, check and improve the software
Analysis and Execution Module	AST analyser, Ruff, Bandit, Radon, Docker sandbox, schema validator, checkpoints	Do the checks that need no LLM and run code safely
Model Access Module	LLM provider, logging proxy	Call the models and record tokens, time and cost
Admin or Researcher Module	SRDD loader, ChatDev adapter, runner, neutral evaluator, statistics	Run benchmarks, ablations, change tests and fault tests and make the reports
Database Module	PostgreSQL, workspace and Git checkpoints	Save all run data and generated files

3.2 Architecture Diagram

MAS is built in layers. The agents are the middle layer and they are made with a LangGraph state graph. The analysis, the execution and the model access are kept in separate layers. This way, the LLM is used only where real thinking is needed.

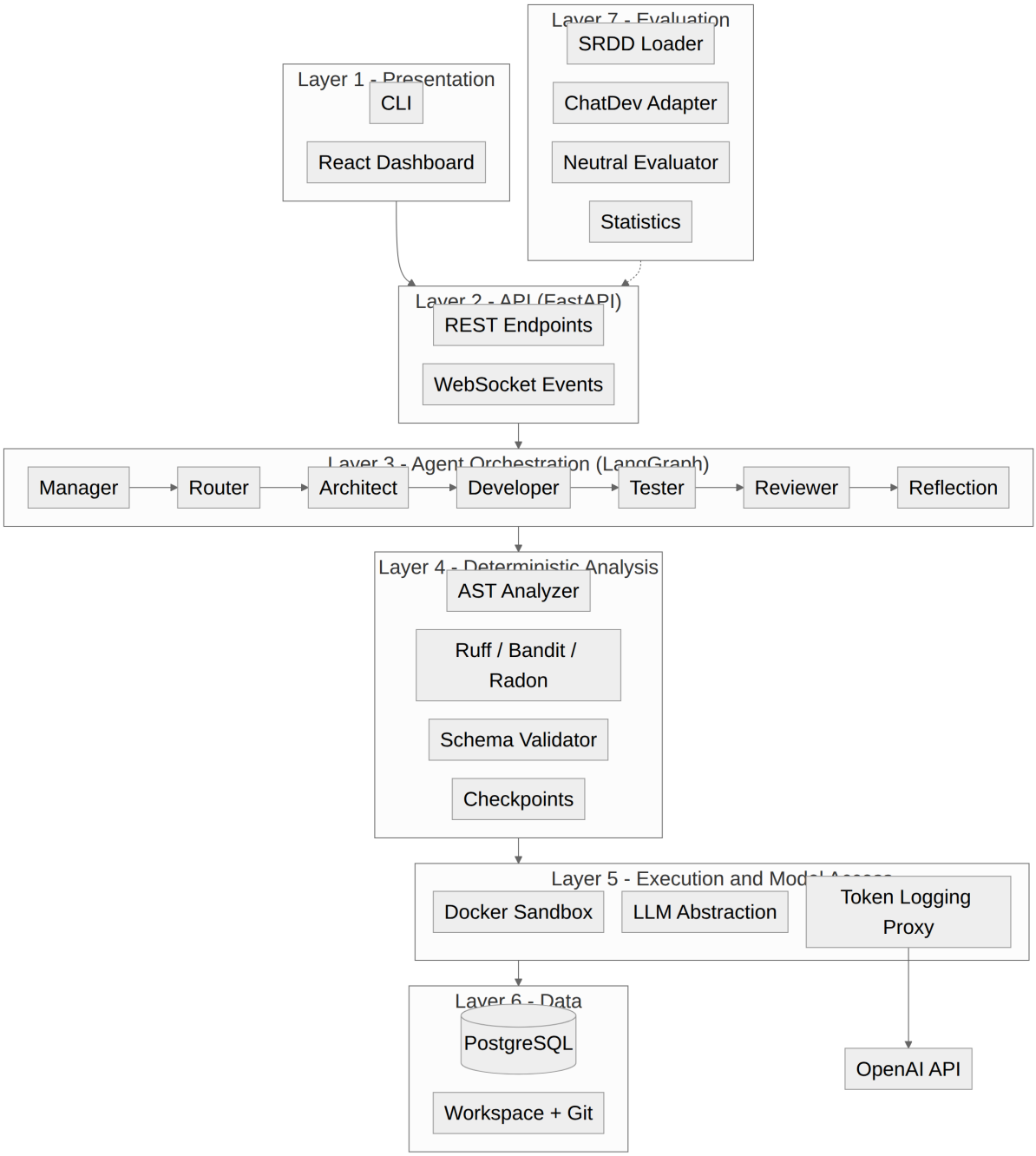


Figure 1: Layered architecture of MAS

3.2.1 Deployment View

Everything runs on one local machine with Docker Compose. The generated code runs only in short-lived sandbox containers. The ChatDev baseline runs in its own container with a fixed version, and it also reaches the model only through our logging proxy.

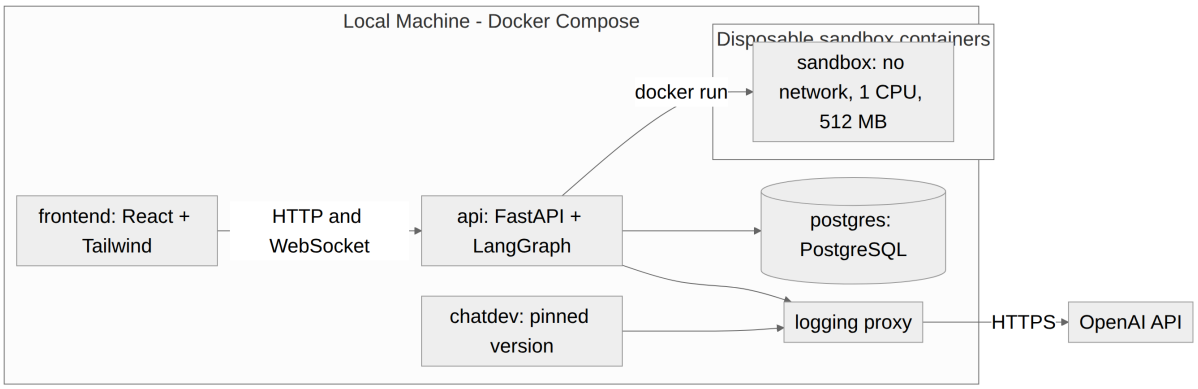


Figure 2: Deployment view (single user, local)

3.3 Data Flow Diagrams

3.3.1 DFD Level 0 (Context Diagram)

At level 0, MAS is shown as one process. It talks to four outside entities: the developer or researcher, the LLM provider, the ChatDev baseline and the SRDD dataset.

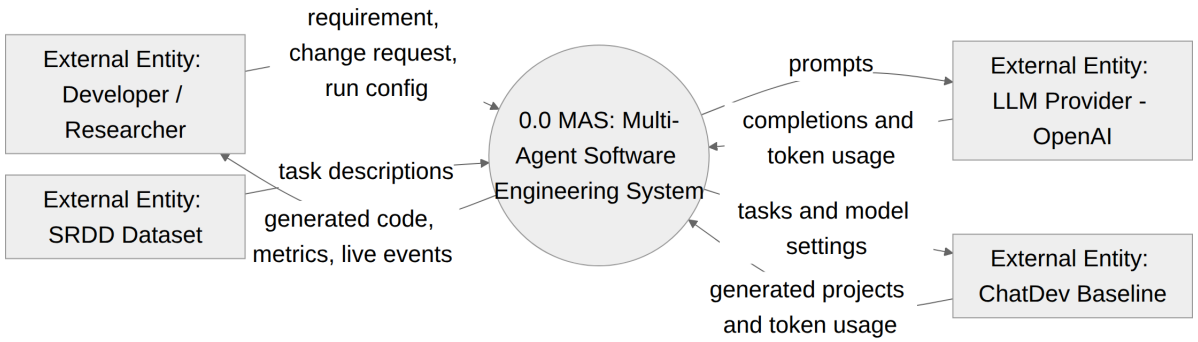


Figure 3: DFD Level 0: context diagram

3.3.2 DFD Level 1

At level 1, the system is split into six processes and four data stores.

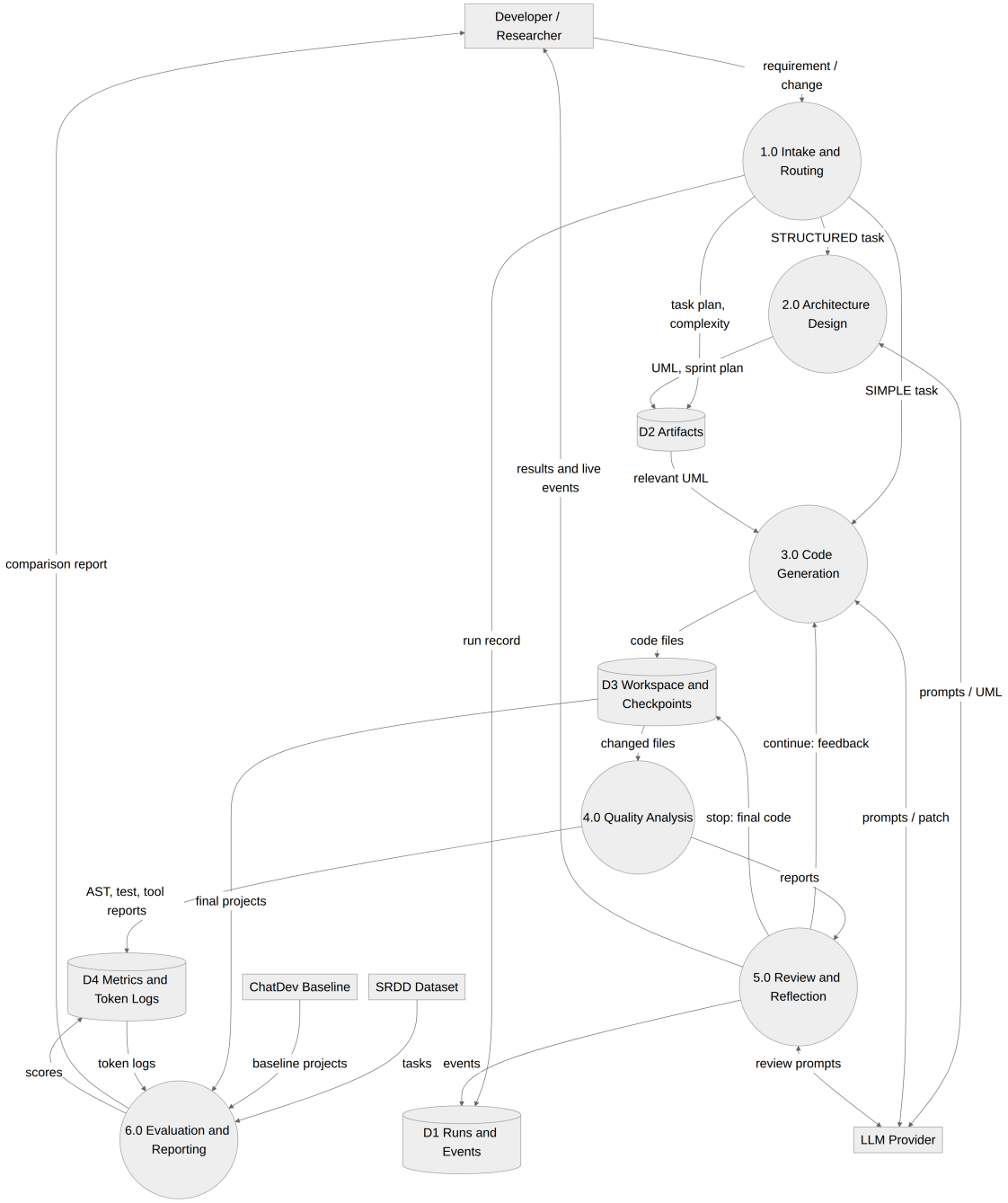


Figure 4: DFD Level 1: main processes and data stores

Process or store	**What it does**
1.0 Intake and Routing	Cleans the requirement, creates the run, finds the complexity and picks the path
2.0 Architecture Design	Makes the UML and the sprint plan for STRUCTURED tasks and checks the output
3.0 Code Generation	Builds a small context, writes a patch, checks it, applies it and saves a checkpoint
4.0 Quality Analysis	Runs the code in the sandbox, AST analysis, MAS pytest, Ruff, Bandit and Radon

Process or store	What it does
5.0 Review and Reflection	Meaning review, score of the round and the continue or stop decision
6.0 Evaluation and Reporting	Runs MAS and ChatDev on SRDD tasks, scores them with the neutral evaluator and finds the statistics
D1 Runs and Events	Run records and the event log
D2 Artifacts	Task plan, user stories, UML, patches, review and reflection outputs
D3 Workspace and Checkpoints	Generated code and Git checkpoints
D4 Metrics and Token Logs	Tool reports, scores, token usage and cost

3.4 Workflows

3.4.1 Overall Workflow

The overall workflow is shown in two parts. Part A is about routing, design and writing the code. Part B is about analysis, review, reflection and the final scoring.

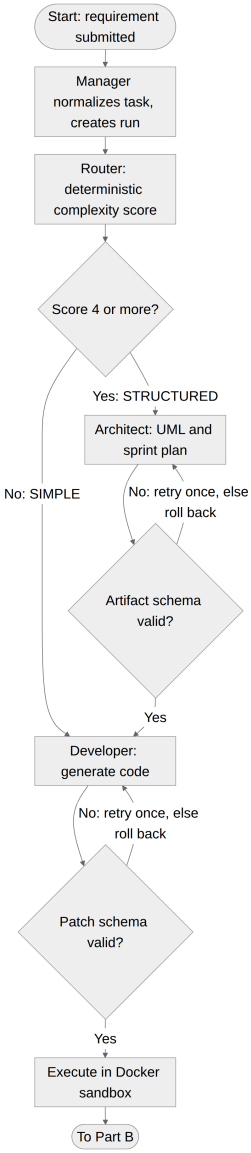


Figure 5: Overall workflow, Part A (routing and code generation)

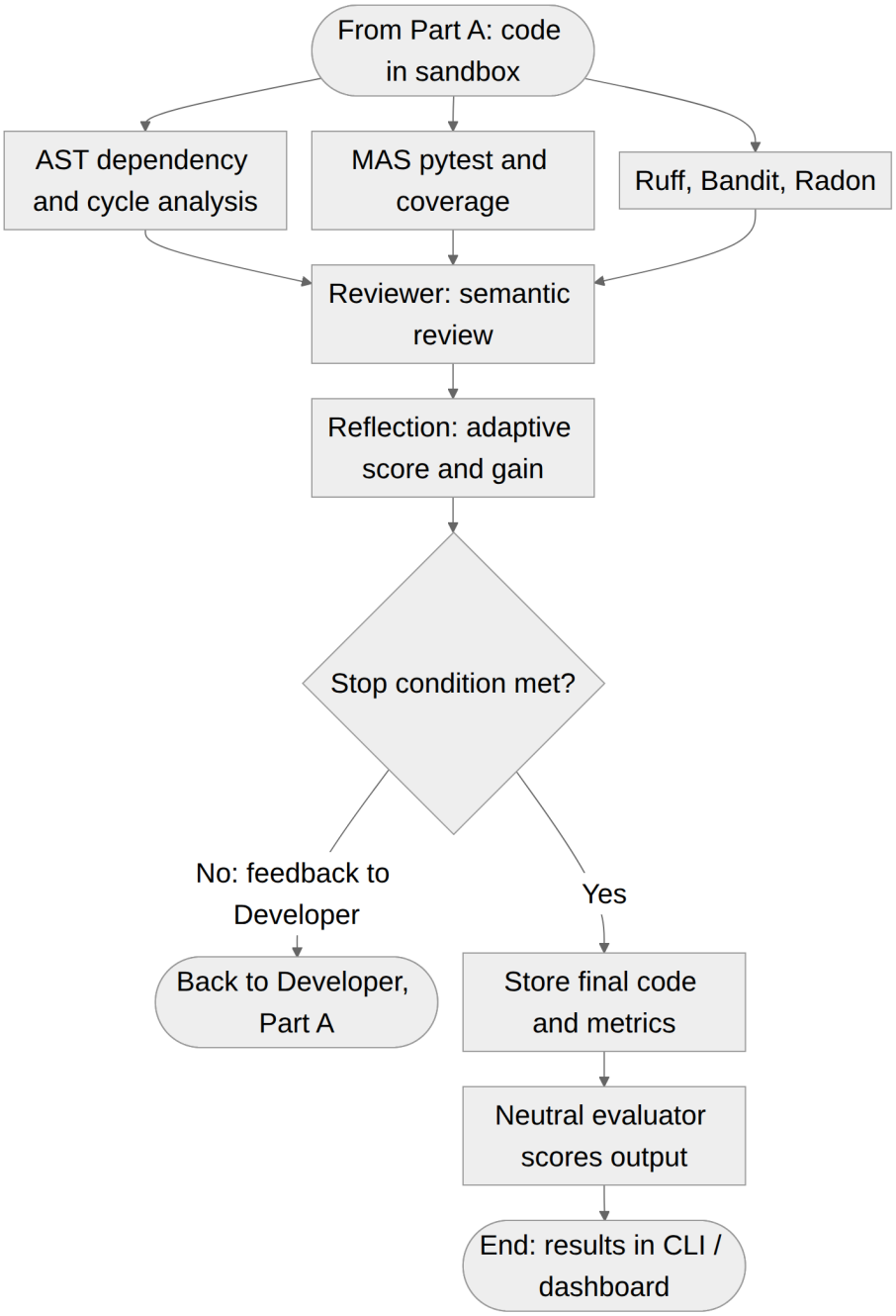


Figure 6: Overall workflow, Part B (analysis, reflection and evaluation)

3.4.2 Individual Workflows

1. **Run Submission Workflow:** Actors: User, API, Manager. The user sends a requirement from the CLI or the API. The API saves the run and its settings and starts the workflow.

2. **Complexity Routing Workflow:** Actor: Router. It applies the keyword and regex rules, finds the score and returns SIMPLE or STRUCTURED along with the matched features. No model call is made.

3. **Architecture Design Workflow:** Actors: Architect, Schema Validator. The Architect

makes the UML and the sprint plan. The validator checks it and it is saved for later use.

4. **Code Generation Workflow:** Actors: Developer, Schema Validator, Checkpoint Manager. The Developer writes a patch from the small context. The patch is checked, the files are written and a checkpoint is saved.

5. **Quality Analysis Workflow:** Actors: Sandbox, AST Analyser, Tester, Quality Tools. The code is run and we get the structure, test, style, security and maintainability reports.

6. **Review and Reflection Workflow:** Actors: Reviewer, Reflection. The Reviewer checks the meaning. Reflection calculates the score and decides to continue or stop.

7. **Change-Request Workflow:** Actors: User, Manager, AST Analyser, Architect, Developer, Tester. MAS finds what changed in the requirement, updates only the affected files, runs the regression tests and finds the code that is not needed any more.

8. **Fault Recovery Workflow:** Actors: Schema Validator, Checkpoint Manager. A wrong artifact is retried once. If it is still wrong, we go back to the last good checkpoint and generate it again.

9. **Benchmark Workflow:** Actors: Researcher, Runner, MAS, ChatDev, Neutral Evaluator, Statistics Engine. Both systems run on the same tasks and repeats, the outputs are scored and the comparison tables are made.

3.5 Architectural Patterns

1. **Layered Pattern:** Presentation, API, agents, analysis, execution and model access, and data are kept in separate layers. Each layer can be changed without touching the others.

2. **Multi-Agent State Graph Pattern:** The agents are the nodes of a LangGraph graph. They read and write one shared state. Conditional edges are used for routing, retry and stopping.

3. **Pipe and Filter Pattern:** The code path goes step by step: Router, Architect, Developer, Analysis, Review and Reflection. Each step uses the checked output of the step before it.

4. **Event-Driven Pattern:** Events of agents and tools are saved in a log and sent over WebSocket. So the page updates without blocking the workflow.

5. **Strategy Pattern:** The LLM provider interface lets us change the model (GPT-4o, GPT-4o-mini or others) just by changing the settings.

6. **Proxy Pattern:** A logging proxy sits between the callers (MAS and ChatDev) and the provider. So the tokens are counted in the same way for both.

7. **Checkpoint Pattern:** Git checkpoints keep the last good state. If an output is wrong, the workflow goes back to it.

8. **Feature Flag Pattern:** Settings switch each saving idea on or off. This is used for the ablation experiments.

4. Non Functional Requirements

These requirements tell how well MAS should work. The numbers given here are targets only. We will confirm them by measuring. They are not results.

Requirement	**Description**
Cost-efficiency	This is the main goal. We measure tokens per task and quality per 1,000 tokens from the usage reported by the provider, and compare with ChatDev and with the MAS ablations. Planning limits: 40,000 tokens per GPT-4o run and 20,000 per GPT-4o-mini run.
Performance	The router decides in milliseconds without any model call. A run can take at most 20 minutes and at most 3 improvement rounds.
Reliability	A wrong artifact is retried once and then rolled back. Failed, cancelled and timed-out runs are logged and kept in the results.
Reproducibility	Fixed seed, temperature 0, saved model version, library versions, Docker image tag and settings for every run. Each task is run 3 times.
Security	Generated code runs in a sandbox with no network and with limits. Secrets stay in environment variables and are never written to generated files or logs. User input is not trusted.
Scalability	Tasks can run in parallel containers. The MVP is for a single local user. Many users is future work.
Maintainability	Agents and tools are separate modules with clear interfaces. Settings are in one versioned file. Pydantic models are the single source for artifacts.
Usability	CLI first, with clear progress output, and one small web page for live view and results.
Portability	A Docker Compose setup that runs the same on Windows, Linux and macOS.

5. Technology Stack Used

1. **Core agent and backend stack**
    - Python 3.11 or above: base language for MAS and for the generated projects
    - LangGraph: state graph that controls the agents
    - FastAPI and Uvicorn: REST and WebSocket interface
    - Pydantic: typed artifacts and JSON Schema
    - PostgreSQL and SQLAlchemy: storage
2. **Model access**
    - OpenAI API with GPT-4o (main) and GPT-4o-mini (cheaper model and reviewer)
    - Local logging proxy that records the usage reported by the provider
3. **Analysis and execution**
    - Python `ast` module: import graph and cycle detection
    - Ruff (style), Bandit (security) and Radon (maintainability)
    - pytest and coverage.py: MAS-internal tests
    - Docker: safe execution of the generated code, with xvfb for programs that open a window
4. **Evaluation stack**
    - SRDD dataset (from the ChatDev project) and ChatDev with a fixed version as the baseline
    - sentence-transformers `all-MiniLM-L6-v2`: embeddings for the consistency score
    - NumPy, pandas and SciPy: metrics and permutation tests
    - Matplotlib: charts for the report
5. **Frontend**
    - React and Tailwind CSS: one run page
    - WebSocket client for live agent events
6. **DevOps and tools**
    - Docker Compose: local deployment
    - Git and GitHub: version control and checkpoints
    - VS Code and Antigravity: development
    - Postman: testing the API

6. Summary

MAS is a cost-aware multi-agent software engineering system. It is designed from the findings of the iCoder paper. It keeps the idea of staged, architecture-aware and review-based code generation, but it makes the costly stages optional and moves the structure checks from LLM calls to normal code. A rule-based router, AST analysis, small context in every round and adaptive stopping reduce the number of tokens. Schema checks, checkpoints and rollback make the system more reliable.

We will test MAS with a repeatable setup. MAS and ChatDev get the same SRDD tasks, the same models and the same scoring scripts. Each task is run three times and the outputs are scored by a hidden neutral evaluator. Along with iCoder's own three metrics, we also measure security, style and maintainability, and we do ablation studies and small change and fault tests. The source code of iCoder is not available, so the comparison with iCoder is indirect and uses only its published numbers. That is why we state our claims as improvements over ChatDev. Human approval gates, large multi-module projects, other programming languages and many-user access control are kept as future work.

References

[1] Y. You, D. Ning, S. Cai, J. Bai, J. Zheng and J. Chen, "iCoder: A multi agent software development platform," *Information and Software Technology*, vol. 198, Article 108224, 2026. DOI: 10.1016/j.infsof.2026.108224.

[2] C. Qian et al., "ChatDev: Communicative agents for software development," arXiv:2307.07924, 2023.